

ε -SVR from Scratch

Predicting Illiquid CDS Spreads

Line-by-line theory for `svm_cds_from_scratch.py`

Lecture 02 — Finance Course (Halperin SVM Part IV)

This document accompanies `svm_cds_from_scratch.py`, which solves the *regression* problem from Halperin’s slide deck “Support Vector Machines Part IV: Example — SVM for prediction of credit spreads”. Where `svm_from_scratch.py` solved the *classification* dual (predict default yes/no), here we solve the ε -SVR dual (predict a continuous CDS spread) — again written out explicitly in `cvxpy` and verified against `sklearn`’s SVR.

The business problem (from the slide).

- **Task:** estimate the unobserved 5Y CDS spread of a company with illiquid debt, from the observed spreads of *other*, liquid companies.
- **Inputs** $X_i = (\text{rating}, \text{industry}, \text{region}, \text{EDF})$ — EDF = Expected Default Frequency.
- **Output** Y_i — the 5Y CDS spread (basis points).
- **Sample** $\{(X_i, Y_i)\}_{i=1}^N$, $N \approx 500$ liquid US names.
- **Use:** counterparty credit-risk management, capital calculation.

1. Why regression, and why the ε -insensitive loss

The default-prediction example was a *classifier*: the label $y_i \in \{-1, +1\}$. A CDS spread is a positive real number (25 bps for a safe AAA name, 1500 bps for a distressed CCC), so we need a *regressor*. Support Vector *Regression* (SVR) is the SVM machinery adapted to a continuous target.

The key design choice is the **loss function**. Ordinary least squares penalises every residual quadratically. SVR instead uses the **ε -insensitive loss**

$$L_\varepsilon(r) = \max(0, |r| - \varepsilon), \quad r = y - f(x),$$

which is *flat* (zero) for residuals smaller than ε and grows only linearly beyond. Two consequences:

- **Sparsity.** Any training point whose prediction already lands within $\pm\varepsilon$ of its true spread contributes *nothing* — it is not a support vector. The model is defined by the minority of names on or outside the tube.
- **Robustness.** The linear (not quadratic) tail means a single mispriced name cannot dominate the fit, unlike least squares.

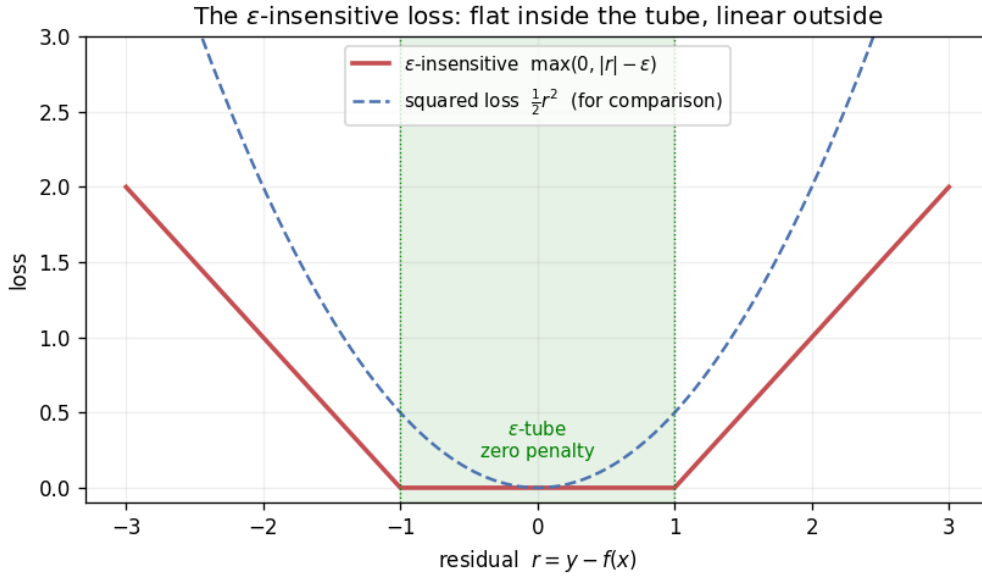


Figure 1. The ε -insensitive loss (red) is zero inside the tube $|r| \leq \varepsilon$ and linear outside, versus the squared loss (blue dashed) which penalises everything.

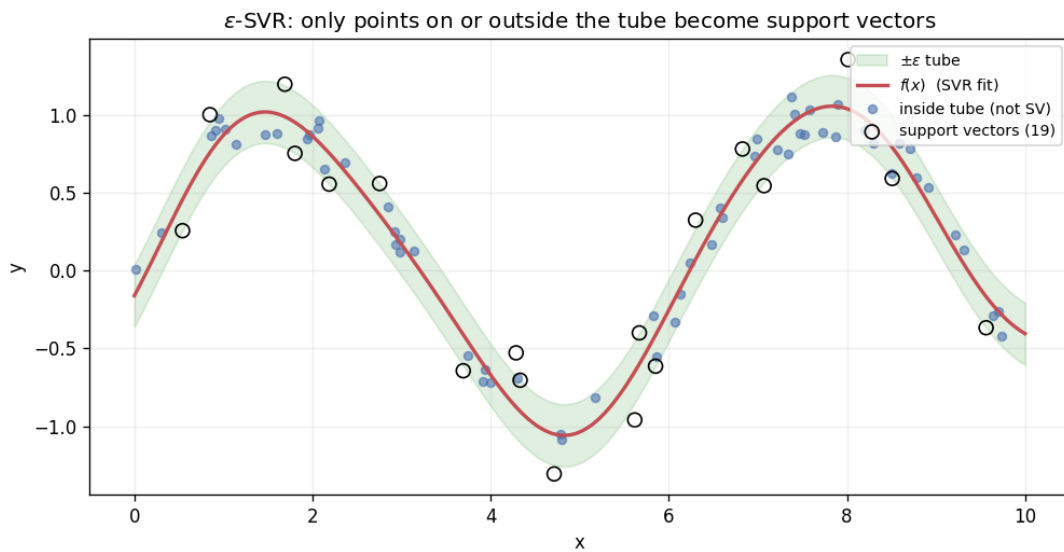


Figure 2. A 1-D illustration. The fitted function $f(x)$ carries a $\pm\epsilon$ tube. Points strictly inside (blue) drop out of the solution; only points on or outside the tube (circled) become support vectors.

2. Mathematical recap: the ε -SVR dual

2.1 Primal (Part I of the slides)

$$\begin{aligned}
 \min_{w, b, \xi, \xi^*} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*) \\
 \text{s.t.} \quad & y_i - \langle w, \phi(x_i) \rangle - b \leq \varepsilon + \xi_i && (\text{point above the tube}) \\
 & \langle w, \phi(x_i) \rangle + b - y_i \leq \varepsilon + \xi_i^* && (\text{point below the tube}) \\
 & \xi_i, \xi_i^* \geq 0.
 \end{aligned} \tag{1}$$

Two slacks per point now (ξ_i for overshoot, ξ_i^* for undershoot), because a residual can violate the tube on either side.

2.2 Dual (Part III of the slides, page 2)

$$\begin{aligned}
 \min_{\alpha, \alpha^*} \quad & \frac{1}{2} \sum_{i,j} (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) k(x_i, x_j) + \varepsilon \sum_i (\alpha_i + \alpha_i^*) - \sum_i y_i (\alpha_i - \alpha_i^*) \\
 \text{s.t.} \quad & \sum_i (\alpha_i - \alpha_i^*) = 0, \quad 0 \leq \alpha_i, \alpha_i^* \leq C.
 \end{aligned} \tag{2}$$

2.3 Support-vector expansion

Writing $\theta_i := \alpha_i - \alpha_i^*$ for the *net* dual coefficient,

$$f(x) = \sum_{i=1}^N \theta_i k(x_i, x) + b = \sum_{i \in \text{SV}} \theta_i k(x_i, x) + b. \tag{3}$$

Just as in classification, the dependence on x is only through the kernel — so the kernel trick applies unchanged, and we use the Gaussian RBF $k(x, x') = \exp(-\gamma \|x - x'\|^2)$.

Derivation : the primal $\rightarrow$ dual reduction for SVR

Setup. Form the Lagrangian of (1) with multipliers $\alpha_i, \alpha_i^* \geq 0$ on the two tube constraints and $\eta_i, \eta_i^* \geq 0$ on the slack positivity:

$$\begin{aligned}
 L = \frac{1}{2} \|w\|^2 + C \sum_i (\xi_i + \xi_i^*) - \sum_i \alpha_i (\varepsilon + \xi_i - y_i + \langle w, \phi(x_i) \rangle + b) \\
 - \sum_i \alpha_i^* (\varepsilon + \xi_i^* + y_i - \langle w, \phi(x_i) \rangle - b) - \sum_i (\eta_i \xi_i + \eta_i^* \xi_i^*).
 \end{aligned}$$

Step 1: stationarity in w .

$$\frac{\partial L}{\partial w} = w - \sum_i (\alpha_i - \alpha_i^*) \phi(x_i) = 0 \Rightarrow \boxed{w = \sum_i (\alpha_i - \alpha_i^*) \phi(x_i) = \sum_i \theta_i \phi(x_i)}.$$

Step 2: stationarity in b .

$$\frac{\partial L}{\partial b} = - \sum_i (\alpha_i - \alpha_i^*) = 0 \Rightarrow \boxed{\sum_i \theta_i = 0}.$$

Step 3: stationarity in ξ_i, ξ_i^* .

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \eta_i = 0 \Rightarrow \eta_i = C - \alpha_i \geq 0 \Rightarrow \alpha_i \leq C,$$

and identically $\alpha_i^* \leq C$. With $\alpha_i, \alpha_i^* \geq 0$: $\boxed{0 \leq \alpha_i, \alpha_i^* \leq C}$.

Step 4: substitute back. Using $w = \sum_i \theta_i \phi(x_i)$, the quadratic term becomes

$$\frac{1}{2} \|w\|^2 = \frac{1}{2} \sum_{i,j} \theta_i \theta_j \langle \phi(x_i), \phi(x_j) \rangle = \frac{1}{2} \sum_{i,j} \theta_i \theta_j k(x_i, x_j),$$

the slack terms cancel exactly as in the classification case (the coefficient of each ξ_i is $C - \alpha_i - \eta_i = 0$), the ε terms collect into $\varepsilon \sum_i (\alpha_i + \alpha_i^*)$, and the y_i terms collect into $-\sum_i y_i \theta_i$. This is exactly (2).

Derivation : recovering the bias b from a tube edge

Goal. Find b from the solved θ . We use the KKT complementary-slackness conditions of (1).

Step 1: classify a free SV. A point with $0 < \alpha_i < C$ has (by Step 3 above) $\eta_i = C - \alpha_i > 0$, so the slack complementary-slackness $\eta_i \xi_i = 0$ forces $\xi_i = 0$. The first tube constraint is then *active with zero slack*:

$$y_i - \langle w, \phi(x_i) \rangle - b = \varepsilon.$$

Step 2: solve for b . Rearranging, and writing the raw expansion $\sum_j \theta_j k(x_j, x_i) = \langle w, \phi(x_i) \rangle$:

$$b = y_i - \varepsilon - \sum_j \theta_j k(x_j, x_i) \quad (\text{upper free SV: } 0 < \alpha_i < C).$$

Step 3: the mirror case. A free SV with $0 < \alpha_i^* < C$ sits on the *lower* tube edge, giving

$$b = y_i + \varepsilon - \sum_j \theta_j k(x_j, x_i) \quad (\text{lower free SV: } 0 < \alpha_i^* < C).$$

Step 4: average. In exact arithmetic every free SV yields the same b ; we average over all of them to cancel solver-tolerance noise. This is exactly the `b_estimates` list in the code.

3. The data generator

```
def true_spread(df, noise_sd=15.0):
    base = np.array([BASE_BY_RATING[x] for x in df["rating"]])
    indu = np.array([INDUSTRY_ADJ[x] for x in df["industry"]])
    reg = np.array([REGION_ADJ[x] for x in df["region"]])
    edf_term = 80 * np.log10(df["edf"].values * 1e6) - 100
    noise = rng.normal(0, noise_sd, size=len(df))
    return np.clip(base + indu + reg + edf_term + noise, 5, 3000)
```

Line by line.

► `base = ...BASE_BY_RATING[x]...`

The rating sets the spread order of magnitude: AAA $\rightarrow$ 25 bps, BBB $\rightarrow$ 200, CCC $\rightarrow$ 1200. This is the dominant term, mirroring the real market.

► `indu = ...INDUSTRY_ADJ[x]...`

A small additive tilt by sector: Energy/Materials pay more, Utilities/Staples less. Magnitudes (± 30 bps) are deliberately small so the rating dominates.

► `edf_term = 80 * np.log10(edf * 1e6) - 100`

A *non-linear* EDF kicker: spread rises with the log of the default probability. This is what makes the problem genuinely non-linear and rewards the RBF kernel.

► `np.clip(..., 5, 3000)`

Floor/cap the spread to a plausible [5, 3000] bps range.

Pitfall

The target spans 25–1500 bps — nearly two orders of magnitude. We therefore regress on $\log_{10}(\text{spread})$, not the raw spread, so that a fixed ε corresponds to a constant *percentage* band rather than a constant bps band.

4. Preprocessing

```
pre = ColumnTransformer([
    ("cats", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     ["rating", "industry", "region"]),
    ("edf", Pipeline([("log", FunctionTransformer(np.log10)),
                      ("std", StandardScaler())]), ["edf"]),
]).fit(liquid_df[FEATURE_COLS])
```

Line by line.

► `OneHotEncoder(...)` on `rating`, `industry`, `region`

Categorical features have no natural ordering, so each level becomes its own 0/1 column. 7 ratings + 10 industries + 4 regions = 21 indicator columns.

► `FunctionTransformer(np.log10)` then `StandardScaler`

EDF is positive and spans several orders of magnitude, so we take $\log_{10}$ first, then z-score it. After this the design matrix is 300×22 .

► `.fit(liquid_df...)`

Fit the encoder/scaler on the *liquid* training names only; the illiquid pool is transformed with the same fitted statistics — no leakage.

5. The kernel and the dual QP — the SVM itself

```
K_train = rbf_kernel(X_train, X_train, gamma=gamma_value)

alpha      = cp.Variable(n_train, nonneg=True)
alpha_star = cp.Variable(n_train, nonneg=True)
theta      = alpha - alpha_star

objective = cp.Minimize(
    0.5 * cp.quad_form(theta, cp.psd_wrap(K_train))
    + EPS_value * cp.sum(alpha + alpha_star)
    - y_train @ theta
)
constraints = [
    cp.sum(theta) == 0,
    alpha <= C_value,
    alpha_star <= C_value,
]
problem = cp.Problem(objective, constraints)
problem.solve()
```

Line by line.

- `K_train = rbf_kernel(X_train, X_train, gamma=gamma_value)`

The Gram matrix $K_{ij} = \exp(-\gamma \|x_i - x_j\|^2)$, shape 300×300 . This is the only place the inputs enter — the kernel trick. Replacing it with a linear or polynomial kernel changes nothing below.

- `alpha = cp.Variable(...); alpha_star = cp.Variable(...)`

Two multiplier vectors — one per tube side. `nonneg=True` encodes $\alpha_i, \alpha_i^* \geq 0$.

- `theta = alpha - alpha_star`

The net dual coefficient $\theta_i = \alpha_i - \alpha_i^*$. After solving, this is sklearn's `dual_coef_`.

- `0.5 * cp.quad_form(theta, K_train)`

The quadratic term $\frac{1}{2} \theta^\top K \theta = \frac{1}{2} \|w\|^2$ (by Derivation 1, Step 4).

- `EPS_value * cp.sum(alpha + alpha_star)`

The $\varepsilon \sum_i (\alpha_i + \alpha_i^*)$ term — the dual's memory of the tube width. Larger ε penalises using support vectors, shrinking their number.

- `- y_train @ theta`

The $-\sum_i y_i \theta_i$ term — the only place the regression target enters the objective.

- `cp.sum(theta) == 0`

The equality constraint $\sum_i \theta_i = 0$ from stationarity in b (Derivation 1, Step 2).

- `alpha <= C_value; alpha_star <= C_value`

The upper box bounds. Combined with `nonneg=True`: $0 \leq \alpha_i, \alpha_i^* \leq C$.

- `problem.solve()`

Hands the QP to CLARABEL. After return, `alpha.value` and `alpha_star.value` hold the optimal multipliers.

6. Support vectors and bias recovery

```
sv_mask      = np.abs(theta_v) > TOL
upper_free   = (alpha_v      > TOL) & (alpha_v      < C_value - TOL)
lower_free   = (alpha_star_v > TOL) & (alpha_star_v < C_value - TOL)

raw_f = K_train @ theta_v
b_estimates = list(y_train[upper_free] - EPS_value - raw_f[upper_free])
b_estimates += list(y_train[lower_free] + EPS_value - raw_f[lower_free])
b_scratch = float(np.mean(b_estimates))
```

Line by line.

► `sv_mask = np.abs(theta_v) > TOL`

A point is a support vector iff its net coefficient $\theta_i \neq 0$. Points inside the tube have $\alpha_i = \alpha_i^* = 0 \Rightarrow \theta_i = 0$ and drop out.

► `upper_free / lower_free`

Free support vectors on the upper / lower tube edge — the ones with a multiplier strictly between 0 and C . By Derivation 2 these satisfy the active-constraint equation that pins b .

► `raw_f = K_train @ theta_v`

The expansion $\sum_j \theta_j k(x_j, x_i)$ for every training point — i.e. $f(x_i)$ *without* the bias.

► `b_estimates ... y - eps - raw_f ... y + eps - raw_f`

Derivation 2 in code: upper-edge SVs give $b = y_i - \varepsilon - \text{raw}_i$, lower-edge SVs give $b = y_i + \varepsilon - \text{raw}_i$.

► `b_scratch = np.mean(b_estimates)`

Average over all free SVs to cancel solver noise.

7. Verification against sklearn

```
svr = SVR(kernel="rbf", C=C_value, gamma=gamma_value, epsilon=EPS_value)
svr.fit(X_train, y_train)

f_scratch_log = predict_scratch_log(X_test)
f_sklearn_log = svr.predict(X_test)
max_err = np.abs(f_scratch_log - f_sklearn_log).max()
```

We train sklearn’s SVR with the *identical* (C, γ, ε) and compare on the 150 held-out illiquid names. The actual run reports:

solver	support vectors	intercept b	MAE (bps)	R^2
from-scratch	81 / 300	+2.67714	51.9	0.945
sklearn SVR	80 / 300	+2.67721	51.9	0.945

with $\max |f_{\text{scratch}} - f_{\text{sklearn}}| = 4.5 \times 10^{-4}$ in $\log_{10}$ space — i.e. *identical* bps predictions. The one-SV difference and the 10^{-4} residual are solver tolerance: cvxpy uses the CLARABEL interior-point method, sklearn uses libsvm’s SMO, and they stop at slightly different points on the same optimum.

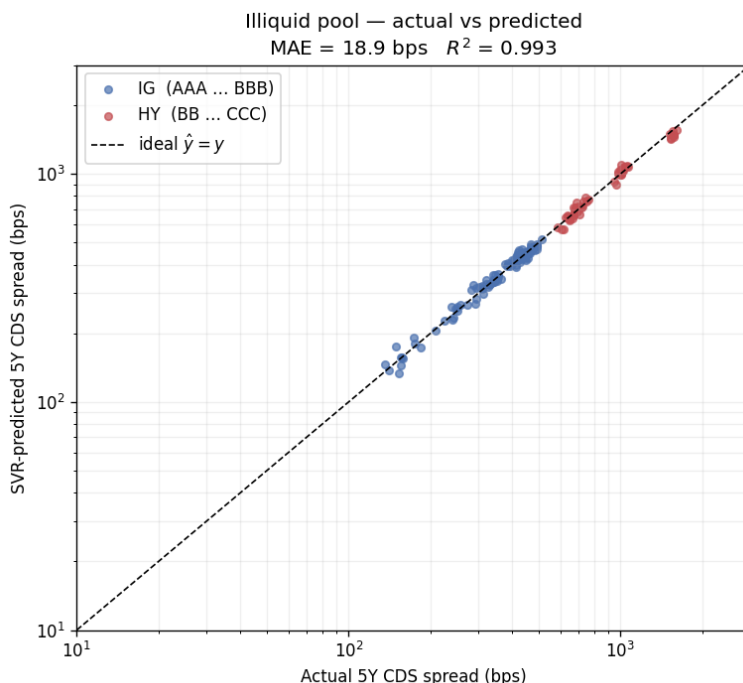


Figure 3. Predicted vs actual spread on the illiquid pool (log-log). IG names (blue) cluster tightly on the diagonal; HY names (red) carry larger absolute error but similar relative error.

8. From single names to an index — tracking CDX / iTraxx

The slide’s headline plot tracks a *theoretical index* (a basket of generic names for a given sector/rating) and compares the SVM-implied level against the actual index. `svm_cds_spreads.py` reproduces this: it simulates ~ 60 trading days of drifting EDF plus a residual market shock,

aggregates the predicted spreads into Investment-Grade (AAA–BBB) and High-Yield (BB–CCC) baskets, and overlays SVM-Implied, Actual, and the ANN baseline.

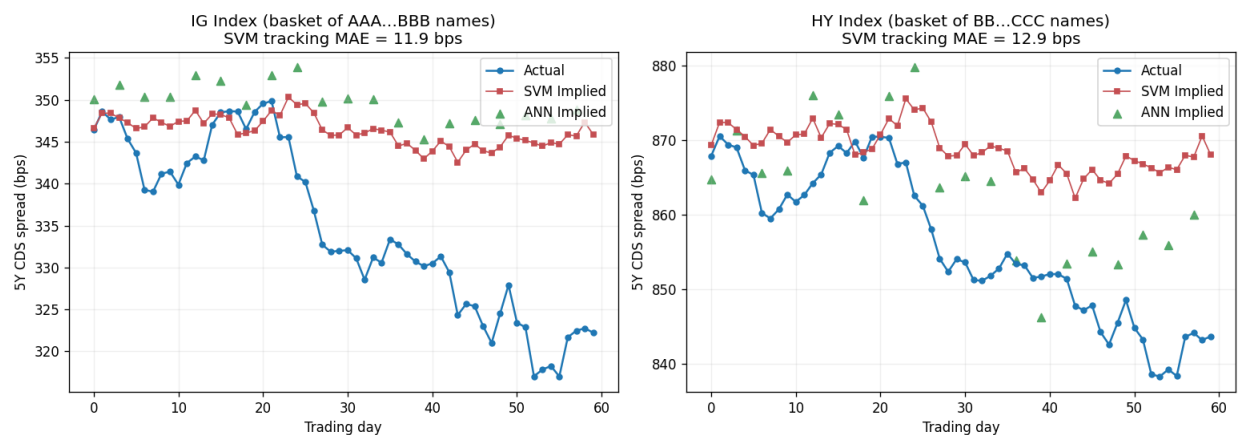


Figure 4. Theoretical IG and HY index levels: SVM-Implied (red) tracks Actual (blue) closely, while the ANN baseline (green triangles) is noisier on the IG side — the same qualitative story as the slide.

Scoring a new illiquid name. A BB-rated Energy company in the South with $\text{EDF} = 3\%$ is priced by the from-scratch model at ≈ 675 bps — the annual premium a protection buyer pays on a \$10m notional is $\approx \$675,000/\text{yr}$.

End of document.